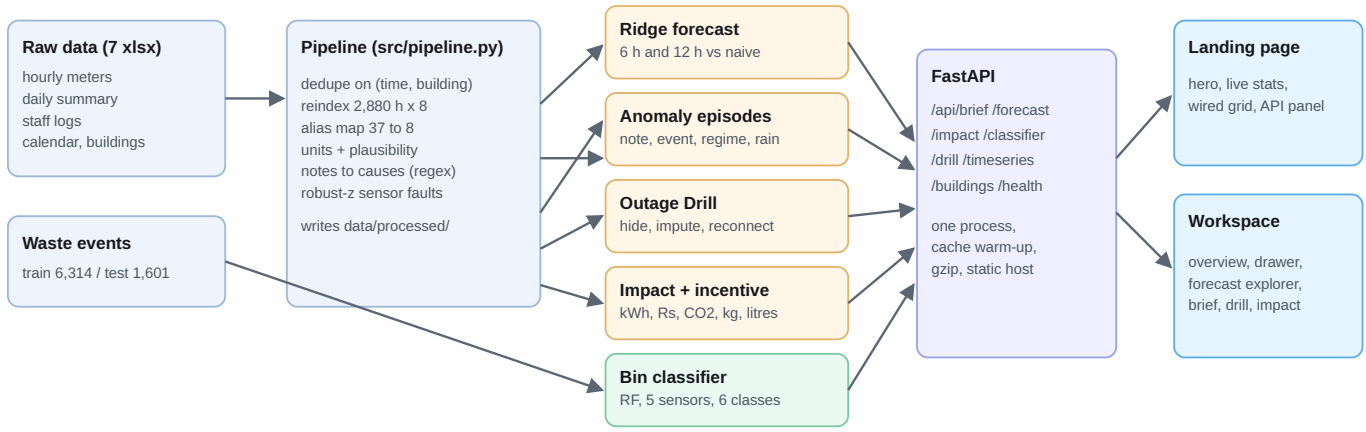
CircuGrid Brief — Technical Report

Predictive Circular-Resource Grid for a Multi-Building Campus · Srujana 2026 “Prompt to Product” · github.com/HARDIKSINGH150206/p2p

CircuGrid Brief fuses four noisy, asynchronous sources from eight campus buildings (hourly meters, daily summaries, staff logs, calendar) plus a smart-bin sensor stream into one grid. It forecasts demand, attributes anomalies to a probable cause with evidence, rehearses a building going offline, classifies waste at the edge, and computes the sustainability impact. Every figure below is produced by the code in the repository on the supplied (synthetic) dataset.

1. System architecture



Component	Responsibility	File
Pipeline	Clean and fuse the sources; keep offline buildings visible; persist processed tables and integrity statistics.	`src/pipeline.py`
Forecast	One ridge regressor per building and horizon (6 h, 12 h) on hour-of-day, day type, weather, current load and footfall, same hour last week and yesterday. Time-ordered split.	`src/forecast.py`
Attribution	Anomaly episodes against the median of days in the same regime, attributed in a fixed order to a staff note, calendar event, regime shift or rain; ranked action cards with evidence lines.	`src/insights.py`
Outage Drill	Hide real hours, estimate from visible hours and a train-only day-type profile, reconnect, measure error vs baselines.	`src/drill.py`
Classifier	Median imputation + 100-tree random forest on five bin sensors, six classes; compost-or-flag routing.	`src/classifier.py`
API + UI	FastAPI serves JSON and the static frontend (no build step): landing page, workspace with overview, forecast explorer, per-building drawer.	`src/api.py`, `web/`

Stack: Python 3.12, pandas 3.1, scikit-learn 1.9, FastAPI/uvicorn, vanilla HTML/CSS/JS. Runs CPU-only; no external API or key is required at runtime. Containerised (`Dockerfile`).

2. Data handling

Seven raw files in `data/` are never modified; cleaned tables are rebuilt into `data/processed/` on every run (git-ignored).

Source	Shape	Role	Issues found
hourly_streams	23,192 × 7	Meters, solar, footfall with arrival time	184 duplicate rows; arrivals up to 12.1 h late; 235 missing hours
daily_summary	960 × 14	Utilities and waste per building-day	nulls: occupancy 5, electricity 11, water 11, wet organic 9 (kept as missing)
manual_logs	415 × 8	Staff readings and free-text notes	37 spellings for 8 buildings; units kWh/units, L/kL, kg/bags; 3 implausible values
calendar	120 × 7	Day type, events, weather	clean
waste_events train / test	6,314 / 1,601	Labelled / unlabelled bin events	~2% nulls per sensor
buildings	8 × 10	Type, solar, composter flag	clean

Cleaning and fusion rules

- **Deduplication and ordering.** Rows are keyed on (timestamp, building) and the latest arrival wins, so retransmissions never double-count; late rows are placed by their capture time, not their arrival time.
- **Intermittent connectivity.** The hourly feed is reindexed to the full 2,880 h × 8 grid, so a building that goes silent stays on the grid as an explicit gap, is flagged `missing`, and short gaps are bridged only for modelling. The Outage Drill rehearses this on recorded data (the data holds no live buffering hardware).
- **Conflicting entries.** A hand-curated alias dictionary resolves all 37 spellings (0 unresolved); a conversion table normalises units; per-stream plausibility ceilings reject unit errors (e.g. a 959,417 value). Manual electricity readings agree with the meter (median ratio 0.99, n = 152).
- **Notes become evidence.** An ordered regex table converts 13 free-text notes into causes (leak, composter, inverter, calibration, meter fault, AC left on, event).
- **Sensor faults are not waste.** A robust z-score flags the B04 reading of 6,234 kWh on 27 Aug; it is linked to the same-day "power analyzer being calibrated" note and excluded from savings.

Data flow

Stage	Input	Transformation	Output
Ingest + clean	raw xlsx	dedupe, reindex, alias, units, outliers	`hourly_clean`, `daily_clean`, `manual_clean`
Fuse	cleaned tables + calendar	join on building and time, regime labels	model features and baselines
Model	fused features	forecast, episodes, drill, classifier	JSON results (cached at start-up)
Serve	results	FastAPI, gzip, health check	landing page and workspace

> **Privacy:** only aggregates are used (occupancy and footfall per building, no individuals). Incentives compare a building with its own baseline so large buildings are not penalised.

3. Models, evaluation and trade-offs

84.96%
bin classifier hold-out accuracy (target >85%), n = 1263

5.21 ms
per event, p50, laptop CPU

5/8
buildings where ridge is best at 6 h

10/10
anomaly episodes attributed

Component	Method and evaluation	Result
Forecast	Train on target times before 18 Sep; score on later days against "same hour last week" and "same hour yesterday". MAE and WAPE per building, horizon and window.	Best of three in 5 of 8 buildings at 6 h; last-week naive wins B04, B06, B08. On the B07 alumni-meet day the yesterday baseline (WAPE 0.3691) beat ridge (0.416).
Outage Drill	B04, 6 h window at 10:00, averaged over 30 test days. The headline imputer never sees the hidden hours.	MAE 1.365 kWh (4.2%) vs 4.283 (last week), 4.757 (unscaled profile), 14.209 (flat mean).
Classifier	Stratified 80/20 split of the labelled file. 5-fold CV on the training split: ≈84.6%; random forest, extra trees and boosting all land near 84.5%, so we did not tune on the hold-out.	84.96% accuracy; per-class recall 0.70 (glass) to 0.90 (metal). Predictions for the 1,601 unlabelled events are in `waste_events_test_predictions.csv`; no accuracy is claimed on them.
Attribution	Ordered rules: note ±1 day → calendar event → regime shift → rain → unexplained. Solar skips events (only weather plausibly explains it).	10 of 10 episodes attributed, each with an evidence line and action.

Accuracy–compute trade-off (edge)

A ridge model per building fits in about 0.01–0.05 s and predicts in microseconds; a gradient-boosting model was timed at roughly 0.14 s per fit on an idle laptop (fit times in the API: ridge 0.003 s, boosting 0.1378 s on this run). Because the forecast is close to persistence at 6 h and ridge already matches or beats the naive baselines in most buildings, the cheaper model is the right edge choice; boosting would cost more without a demonstrated gain.

4. Scale, cost, failure modes

From 8 to 30+ buildings

- **Alias table** filled at onboarding; unmatched names go to a review queue.
- **One small model per building;** cold start uses the day-type profile for about two weeks.
- **Late-arrival replay** bounded per building (here ≤ 12 h).
- **Central roll-up** of Brief rows only; raw streams stay on the edge box.

Per-building hardware: ≈ Rs 12,000 (ESP32 + meter 3,500; two-bin sensor kit 3,000; Pi-class box 5,000; misc 500) — a planning assumption, not a quote.

Failure modes and mitigations

- **Sensor drift or fault.** Robust outlier flag cross-checked against notes; flagged readings excluded from forecasts and savings (demonstrated on B04, 27 Aug).
- **Gaming the incentive.** Savings computed from meters, not logs; per-building baseline.
- **Model degradation over a semester.** Rolling baseline reset after exam or break regimes; error tracked against the naive baseline.

> **Known limitations:** synthetic data; classifier is 84.96%, just under 85%; latency measured on a laptop, not edge hardware; manual numeric readings are cleaned but not yet fused; the incentive loop is a baseline ranking with no behaviour change measured; scale claims are a design argument, not load-tested.